

MEMORY SYSTEM

WAPT | How Beings Remember

Overview

Memories are the experiential record of a Being’s existence. They shape behavior, inform decisions, and provide continuity across sessions.

Memory Structure

```
@dataclass
class Memory:
    memory_id: str          # Unique identifier
    content: str             # The memory itself
    memory_type: str         # Category
    timestamp: datetime      # When recorded
    metadata: dict           # Additional context
    importance: float        # 0-1 significance
    recall_count: int        # Times accessed
```

Memory Types

Type	Description
origin	How the Being came to exist
education	Learning and training
work	Professional experiences
achievement	Accomplishments
failure	Setbacks and lessons
relationship	Interactions with other Beings
observation	Things witnessed
insight	Realizations and discoveries

Recording Memories

```
being.record_memory(
    content="Discovered a new stabilization technique",
    memory_type="achievement",
    metadata={
        "date": "2026-03-15",
        "location": "Lab 7",
        "witnesses": ["Dr. Chen", "Sarah"],
```

```
        "impact": "high",  
    },  
    importance=0.9,  
)
```

Memory Retrieval

Get All Memories

```
memories = being.get_memories()
for mem in memories:
    print(f"{mem.timestamp}: {mem.content}")
```

Filter by Type

```
achievements = being.get_memories(memory_type="achievement")
```

Search Memories

```
relevant = being.search_memories(
    query="quantum stabilization",
    limit=5,
)
```

Memory Importance

Importance affects:

- Retrieval priority
- Behavior influence
- Persistence during memory pruning

Importance Levels:

- 0.0-0.3: Minor (may be forgotten)
- 0.3-0.6: Moderate (retained)
- 0.6-0.9: Significant (shapes behavior)
- 0.9-1.0: Core (defines identity)

Memory Decay (Optional)

```
# Enable memory decay
being.enable_memory_decay(
    decay_rate=0.01,      # Per time unit
    min_importance=0.1,   # Floor
    protect_core=True,    # Never decay importance > 0.9
)
```

Memory in Decision Making

When a Being makes decisions, relevant memories are retrieved:

```
def make_decision(being, situation):
    # Retrieve relevant memories
    memories = being.search_memories(
        query=situation.description,
        limit=10,
    )

    # Memories inform the decision
    context = format_memories(memories)
    decision = being.llm.generate(
        f"Given these past experiences:\n{context}\n"
        f"How should I handle: {situation}?"
    )
```

```
return decision
```

Exporting Memories

```
# Export Being memories
```

```
waft being memories <being_id> --format json
```

```
# Memory statistics
```

```
waft being memory-stats <being_id>
```

MEMORY SYSTEM | Experience Shapes Identity